Table 5. Prediction accuracy as a function of the size of the training set

Training/validation/test proportion (%)	Prediction accuracy (%)
60/20/20	99.7
5/5/90	99.6
1/1/98	99.3
0.5/0.5/99	98.9
0.1/0.1/99.8	92.2

the two equations, and the output of the CNN was a yes/no label depending on whether p implies q. The CNN processed the input data using a 5-layer architecture, each layer composed of a 1-D convolution, followed by batch normalization and rectified linear unit activation functions [27]. After the last convolutional layer, a flattening layer and a softmax activation function were used to obtain the output of the network, i.e., the prediction of the implication for the input pair of equations. Note that we trained our models with the 16 October 2024 version of the (infinite) implication graph, for which 362 of the hardest implications (less than 0.002% of the total) were still unknown.

Prior to training the CNN, we divided the data into training (60%), validation (20%), and test (20%) subsets. The CNN was implemented in TensorFlow 2.9.0 [1] and trained on an NVIDIA 3080 Ti GPU with the following configuration: the binary cross-entropy as the loss function to minimize, the Adam method with an initial learning rate of 10^{-3} for the adjustment of network weights, a batch size of 1024 with random shuffling, a learning rate reduction by a factor of 2 after 15 epochs without improvement in the validation loss, and early stopping if no improvement occurred for 40 epochs, with the model with the lowest validation loss being retained as the final CNN. The final CNN was evaluated on the test set, reaching a prediction accuracy of 99.7%, which means that the model misclassified around 66k of the 22 million implications. Since this accuracy was somewhat surprising for such a small and "simple" model, as a control we generated a random label (yes/no) for each pair of equations, then trained the CNN on this data with 60%/20%/20% training/validation/test percentages, resulting in a 49.99% accuracy, as expected from an unbiased model.

It could be the case that the high accuracy of our CNN model was mostly due to it learning the transitivity of the implication relation, as opposed to it discovering patterns in the identities. To clarify this point, it was proposed to train our CNN model either on a random poset or on the equational poset with vertices and labels permuted, and check whether a similar accuracy was achieved. This experiment has not been performed yet.

In any case, since there are around 600k explicit implications from which the rest can be derived by transitivity (2.7% of the total), if the CNN was learning transitivity it should perform well with a very small training dataset. Accordingly, we trained and assessed a CNN model with a 5%/5%/90% training/validation/test proportion, with a resulting 99.6% accuracy on test (the process finishing in under 20 minutes). But the high accuracy is maintained with even smaller training datasets, as evidenced in Table 5.

Since these training datasets were significantly smaller than the subset of explicit implications, and were not carefully chosen from the poset extremes but taken randomly, we can

Table 6. Sizes of different encodings for the implication graph

Encoding model	Uncompressed size	Compressed size
Labelled pairs of equations	1.5 GB	9 MB
Bit table	42 MB	40 kB
CNN (99.7% accuracy)	1.34 MB	700 kB

conclude that even if the CNN were learning transitivity, that by itself is probably insufficient to explain the high accuracy achieved by the CNN model.

Since sometimes machine learning is announced as a form of data compression, let us now comment on the level of data compression achieved by our CNN model. In Table 6 we compare the sizes of three different encodings of the implication graph: a) As the simplest approach, we can encode the full implication graph in one file as labelled pairs of equations of the form (p, q, yes/no). b) On the other extreme, we can encode it as a bit table containing neither the explicit equation expressions nor their numbers, but just a 1/0 label for each point with coordinates (p,q), together with a table mapping each number to its corresponding equation expression and a small script to recover the file in a). c) Finally, we can encode it in the complete files of the CNN model produced by TensorFlow 2.9.0. In addition, we can either consider these files in their raw form, or we can highly (but losslessly) compress them to achieve a rough comparison of their actual information content; accordingly, in Table 6 we also include the sizes of the encoding models when compressed with 7-zip LZMA2 ultra compression with a 1536MB dictionary size and a 273 word size.

As we see, the information in the CNN model is more than 13 times less than in the labelled pairs model, while it is more than 18 times that of the bit table model. We also note that the CNN model in its raw form is already quite incompressible, and more than 30 times smaller than the raw bit table.

Lastly, note that our CNN model does not only encode the implication graph up to order 4 (with 0.03% of noise), but a priori may also be able to predict it for higher orders with a significant accuracy. Thus it could be used to guide and speed up the determination of the implication graph up to order 5, by letting ATPs focus first on the CNN's predicted status of the studied implication.

12. User Interfaces

A number of custom web applications were developed as part of the ETP. While many past Lean formalization projects have primarily relied on the Lean blueprint tool to organize tasks and track progress, the large volume of (transitive) implications tracked by the ETP, along with the research-oriented nature of the project, necessitated the development of custom tools to complement the blueprint tool. These web applications also made information more accessible to project participants and other interested parties, including those unfamiliar with Lean or the custom software developed for the project. The project features four primary interfaces:

(1) The **ETP dashboard**[28] displays the high-level overview of the project: the total number of resolved, conjectured, and unknown implications for the general and finite implication graphs. The dashboard also includes links to other tools, data, and visualizations about the implication graphs.
(2) The **Equation Explorer**[29] is the primary tool to navigate the implication graph. For a given equation, it displays its inbound and outbound implications, as well as other members of its equivalence class. The explorer allows navigating either the general or finite implication graphs. The explorer also features custom commentary for a given equation (when available), serving as a repository for information and links. It also links to Graphiti visualizations and an example of its smallest satisfying magma, if one exists. Figure 16 shows an example view of the explorer.
(3) **Graphiti**[30] visualizes the implication graph as a Hasse diagram, where downward edges represent subset relationships, and upward edges represent implications. Equivalence classes are collapsed into single nodes for clarity. Graphiti supports search parameters to visualize specific subsets of the graph. It can also display the entire implication graph, though the complete graph is large and challenging to navigate. Figure 9 is an example of a Graphiti visualization.
(4) The **Finite Magma Explorer**[31] tests which equations a given finite magma satisfies or fails to satisfy. Users input finite magmas as Cayley tables. The tool is aware of the finite implication graph, so if an input magma witnesses an unknown refutation, it notifies the user and provides instructions for contributing it to the GitHub repository.

The data for these tools is extracted directly from the Lean-formalized proofs in the project's GitHub repository, ensuring it always faithfully reflects the current state of progress. Additionally, the data is automatically updated with each code change using continuous integration (CI), eliminating the need for manual updates.

13. Data management

All the data and formalizations generated by the project are placed in the GitHub repository[32], while the discussion is almost entirely contained in a dedicated channel on the Lean Zulip[33]. The implication graph can be downloaded from Equation Explorer[34], and can also indicate the individual *Lean* theorems required to establish or refute any given implication, although currently we have only formalized a generating set of implications and refutations in *Lean*, rather than the entirety of the implication graph.

[28] `https://teorth.github.io/equational_theories/dashboard/`
[29] `https://teorth.github.io/equational_theories/implications/`
[30] `https://teorth.github.io/equational_theories/graphiti/`
[31] `https://teorth.github.io/equational_theories/fme/`
[32] `https://github.com/teorth/equational_theories`
[33] `https://leanprover.zulipchat.com/#narrow/channel/458659-Equational`
[34] `https://teorth.github.io/equational_theories/implications/`